

CONTINUATION PROGRAMMING

ISIS 4217

CAN ALL RECURSIVE CALLS BE TAIL RECURSIVE



Well…. it turns out **you** actually **can**

… it’s just not straightforward though

From recursive to iterative processes

Factorial

```
fun {Fact N}
   fun {FactIter I F}
      if I == N then F
      else {FactIter I+1 F*(I+1)}
      end
   end
   {FactIter 0 1}
end
```

From recursive to iterative processes

Factorial

```
fun {Fact N}
   fun {FactIter I F}
      if I == N then F
      else {FactIter I+1 F*(I+1)}
      end
   end
   {FactIter 0 1}
end
```

What happens when there are multiple things to “return”?

Accumulators

Accumulators are used to deal with state in declarative programs, passing the "state" as function arguments

```
fun {F P1 P2 A}
```

Whenever the state is composed of several independent components, procedures assume each component

$$\texttt{State} = (I_1\ O_1\ I_2\ O_2\ \ldots\ I_n\ O_n)$$

```
fun {F P1 O1 P2 O2}
```

Accumulators

Merge sort

We have to accumulate the list we are sorting

```
fun {MergeSort Xs}
 {IterMergeSort {Length Y} Xs Xr Ys}
 Ys
end
```

Accumulators

Merge sort

We have to accumulate the list we are sorting

```
fun {MergeSort Xs}
 {IterMergeSort {Length Y} Xs Xr Ys}
 Ys
end
```

size of list to sort

Accumulators

Merge sort

We have to accumulate the list we are sorting

```
fun {MergeSort Xs}
 {IterMergeSort {Length Y} Xs Xr Ys}
 Ys
end
```

list to sort

size of list to sort

Accumulators

Merge sort

We have to accumulate the list we are sorting

```
fun {MergeSort Xs}
 {IterMergeSort {Length Y} Xs Xr Ys}
 Ys
end
```

list to sort

size of list to sort

remainder of the list to sort

Accumulators

Merge sort

We have to accumulate the list we are sorting

```
fun {MergeSort Xs}
 {IterMergeSort {Length Y} Xs Xr Ys}
 Ys
end
```

list to sort

sorted list

size of list to sort

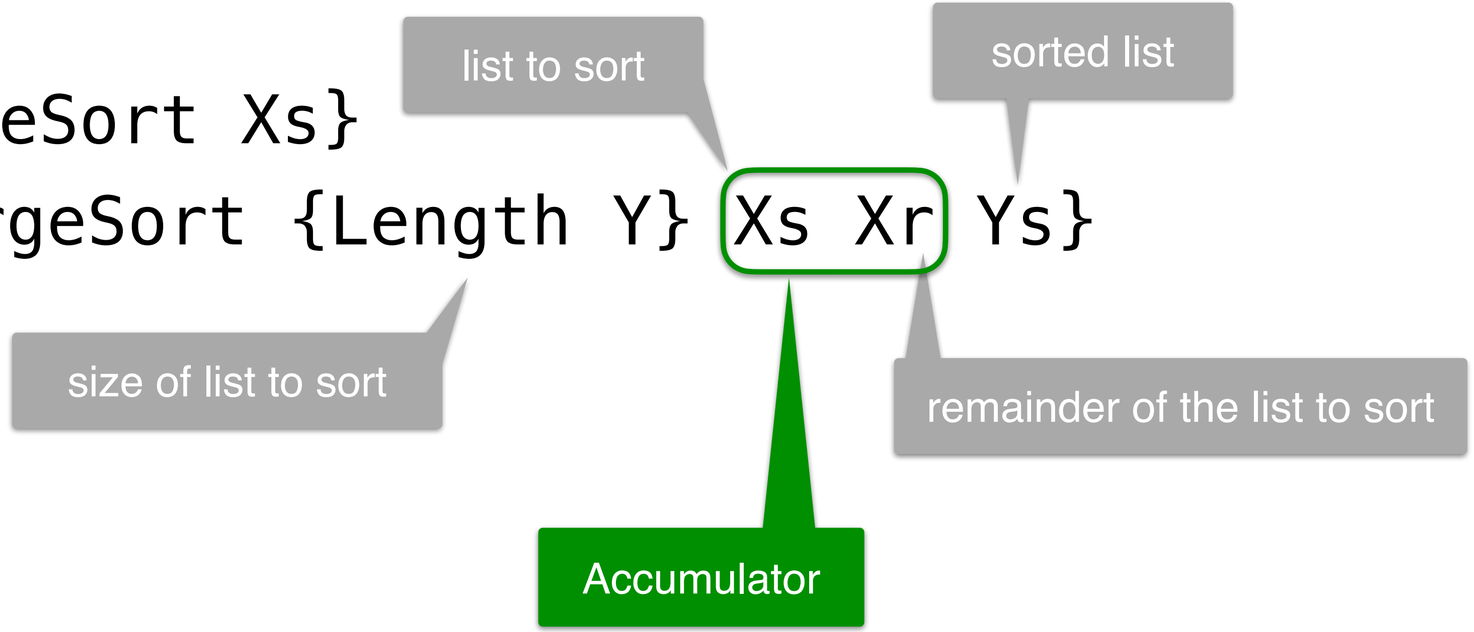
remainder of the list to sort

Accumulators

Merge sort

We have to accumulate the list we are sorting

```
fun {MergeSort Xs}
 {IterMergeSort {Length Y} Xs Xr Ys}
 Ys
end
```



Accumulators

Merge sort

```
proc {MergeSortAcc N S0 S ?Xs}
   if N==0 then S = S0 Xs = nil
   elseif N ==1 then
      S0 = X|S
      L = [X]
   else
      NL = N div 2
      NR = N - NL
      {MergeSortAcc NL S0 S1 Xs1}
      {MergeSortAcc NR S1 S Xs2}
      Xs = {Merge Xs1 Xs2}
  end
end
```

Accumulators

Merge sort

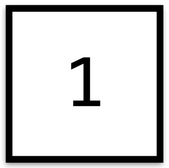
```
proc {MergeSortAcc N S0 S ?Xs}
   if N==0 then S = S0 Xs = nil
   elseif N ==1 then
      S0 = X|S
      L = [X]
   else
      NL = N div 2
      NR = N - NL
      {MergeSortAcc NL S0 S1 Xs1}
      {MergeSortAcc NR S1 S Xs2}
      Xs = {Merge Xs1 Xs2}
  end
end
```

Multiple accumulators

Let’s make a binary expression evaluation stack

Multiple accumulators

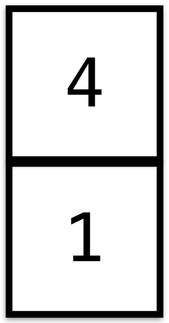
Let’s make a binary expression evaluation stack




```
push(1)
```

Multiple accumulators

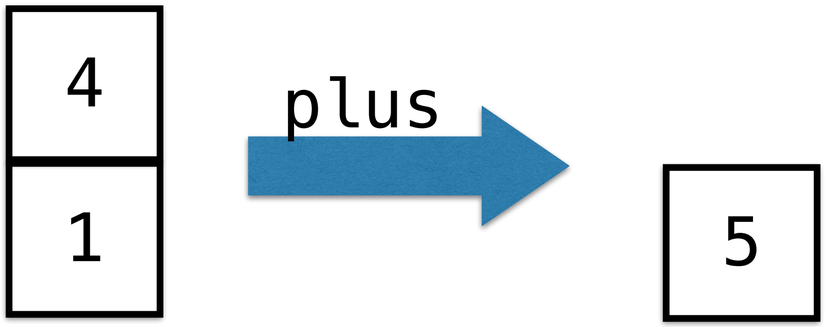
Let’s make a binary expression evaluation stack




```
push(1)
push(4)
```

Multiple accumulators

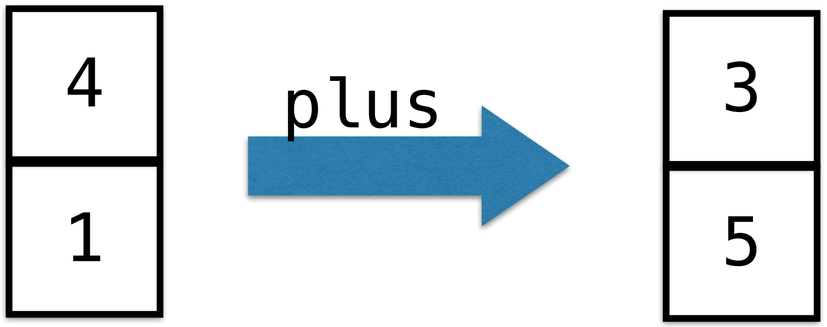
Let’s make a binary expression evaluation stack




```
push(1)
push(4)
```

Multiple accumulators

Let’s make a binary expression evaluation stack

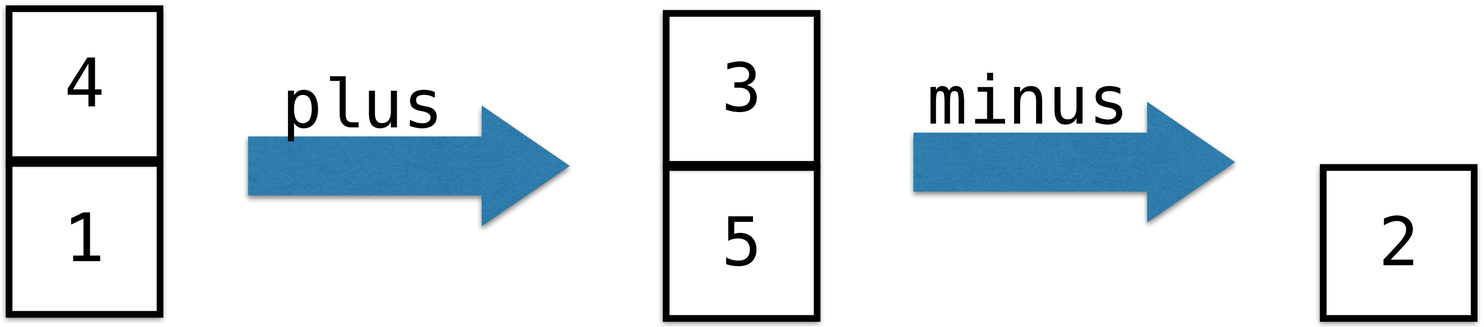



```
push(1)
push(4)
```

```
push(3)
```

Multiple accumulators

Let’s make a binary expression evaluation stack




```
push(1)
push(4)
```

```
push(3)
```

Multiple accumulators

Expression stack

Expressions can be represented as a binary tree (of tuples)

```
minus(plus(1 4) 3)
```

Let's write a procedure that:

- takes an arithmetic expression
- translates it into a stack
- counts the number of instructions "evaluated"

Recursive?
What are the accumulators?

Multiple accumulators

Expression stack

```
fun {ExprCode Expr}
   case Expr
   of plus(Expr1 Expr2) then
     1 + {ExprCode Expr1} + {ExprCode Expr2}
   [] minus(Expr1 Expr2) then
     1 + {ExprCode Expr1} + {ExprCode Expr2}
   end
end
```

Multiple accumulators

Expression stack

and the stack?

```
fun {ExprCode Expr}
   case Expr
   of plus(Expr1 Expr2) then
      1 + {ExprCode Expr1} + {ExprCode Expr2}
   [] minus(Expr1 Expr2) then
      1 + {ExprCode Expr1} + {ExprCode Expr2}
   end
end
```

Multiple accumulators

Expression stack

```
fun {ExprCode Expr}
   case Expr
   of plus(Expr1 Expr2) then
      plus|{Append {ExprCode Expr1} {ExprCode Expr2}}
   [] minus(Expr1 Expr2) then
     minus|{Append {ExprCode Expr1} {ExprCode Expr2}}
   [] I andthen {IsInt I} then
     push(I)|nil
   end
end
```

Multiple accumulators

Expression stack

and the count?

```
fun {ExprCode Expr}
   case Expr
   of plus(Expr1 Expr2) then
      plus|{Append {ExprCode Expr1} {ExprCode Expr2}}
   [] minus(Expr1 Expr2) then
     minus|{Append {ExprCode Expr1} {ExprCode Expr2}}
   [] I andthen {IsInt I} then
     push(I)|nil
   end
end
```

Multiple accumulators

Expression stack

```
proc {ExprCode Expr Stack Count}
   case Expr
   of plus(Expr1 Expr2) then
     Stack = plus|{Append {ExprCode Expr1 Stack Count+1}
                          {ExprCode Expr2 Stack Count+1}}
   [] minus(Expr1 Expr2) then
     Stack = minus|{Append {ExprCode Expr1 Stack Count+1}
                           {ExprCode Expr2 Stack Count+1}}
   [] I andthen {IsInt I} then
     Stack = push(I)|nil
     Count = 1
   end
end
```

Multiple accumulators

Expression stack

```
proc {ExprCode Expr C0 C N0 N}
   case Expr
   of plus(Expr1 Expr2) then C1 N1 in
      C1 = plus|C0
      N1 = N0 + 1
      {SeqCode [Expr2 Expr1] C1 C N1 N}
   [] minus(Expr1 Expr2) then C1 N1 in
      C1 = minus|C0
      N1 = N0 + 1
      {SeqCode [Expr2 Expr1] C1 C N1 N}
   [] I andthen {IsInt I} then
      C = push(I)|C0
      N = N0 + 1
   end
end
```

Multiple accumulators

Expression stack

```
proc {SeqCode Xs C0 C N0 N}
   case Xs of nil then
    C = C0
    N = N0
    [] X|Xr then N1 C1 in
       {ExprCode X C0 C1 N0 N1}
       {ExprCode Xr C1 C N1 N}
   end
end
```

Multiple accumulators

Fibonacci

Do it yourself

Multiple accumulators

Fibonacci

Do it yourself

Multiple accumulators

Fibonacci

Do it yourself

```
proc {FibAcc N L R}
   if N == 0 then L + R
   else
      {FibAcc N-1 R L+R}
   end
end
```

Continuations

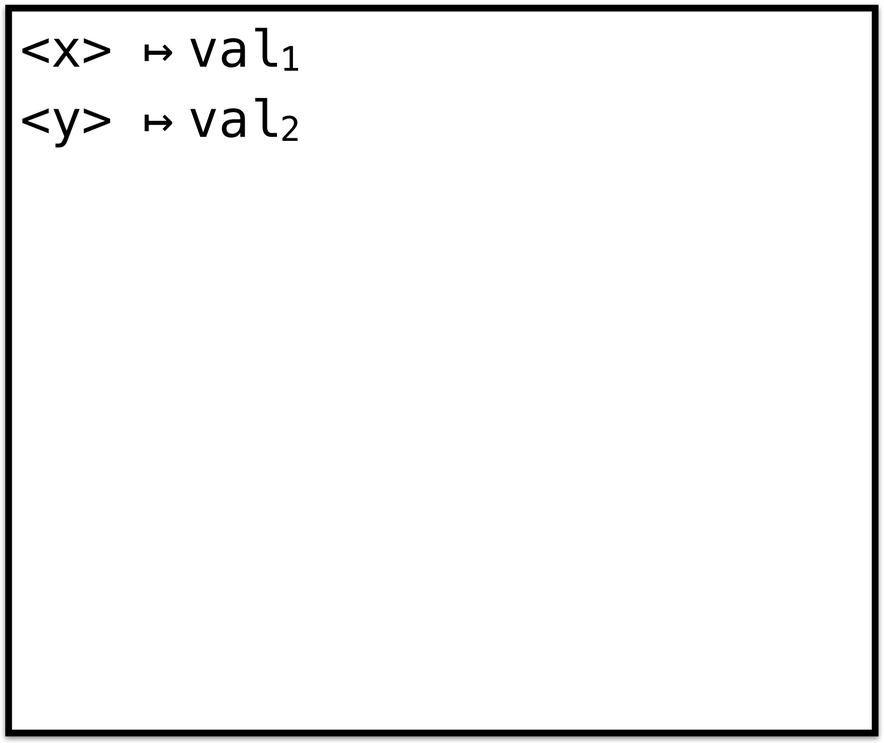
Continuations are a programming technique to express “**the rest of the program**,” similar to callbacks or goto calls

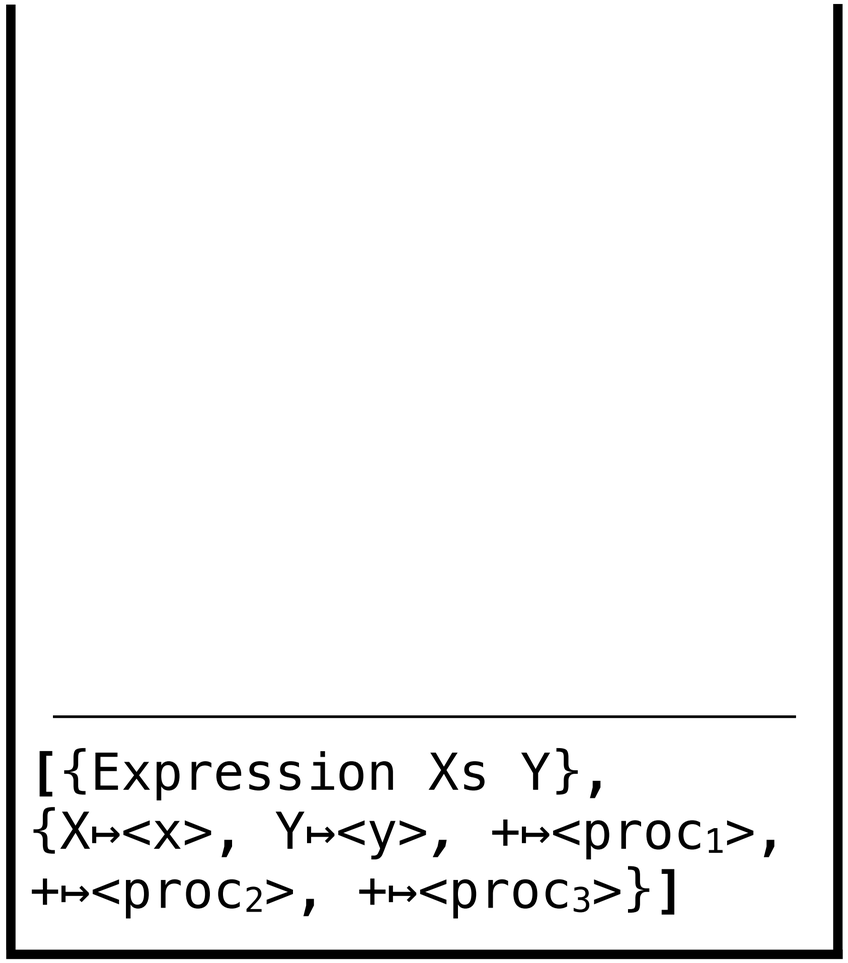
Continuations are normally used by compilers to **optimize** your program during program transformation

Continuation passing stile is a higher-order feature to program with continuations

Continuations in function evaluation

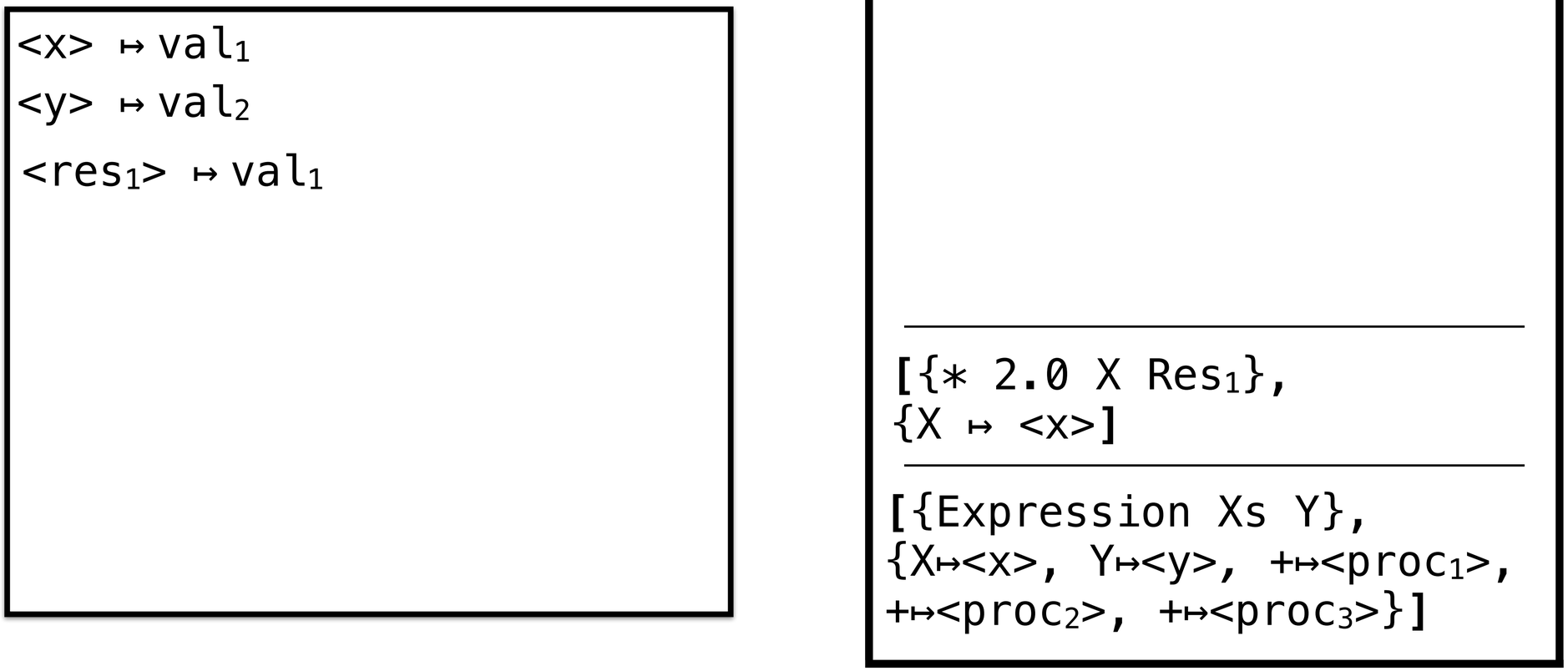
```
fun {Expression X Y}
   2.0*X + 3.0*Y + 1.0 div X + 2.0 div Y
end
```





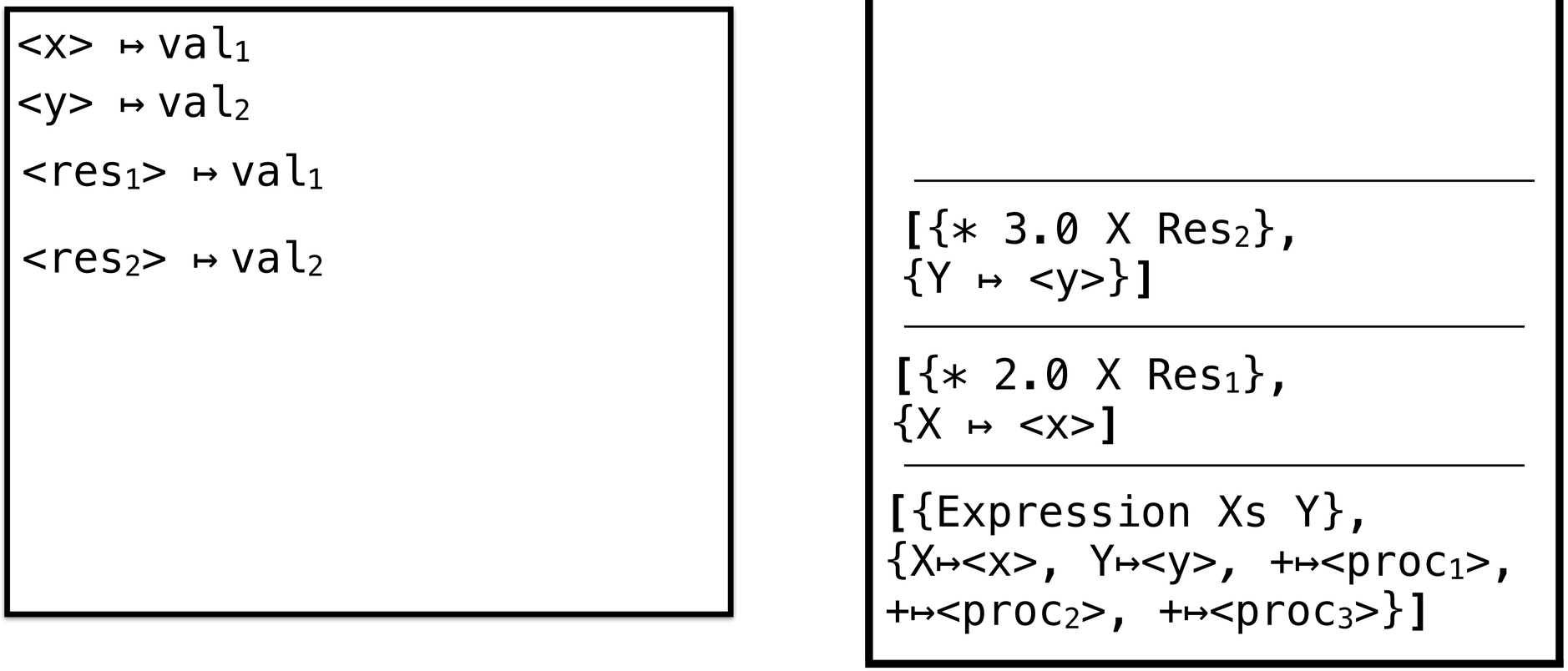
Continuations in function evaluation

```
fun {Expression X Y}
   2.0*X + 3.0*Y + 1.0 div X + 2.0 div Y
end
```



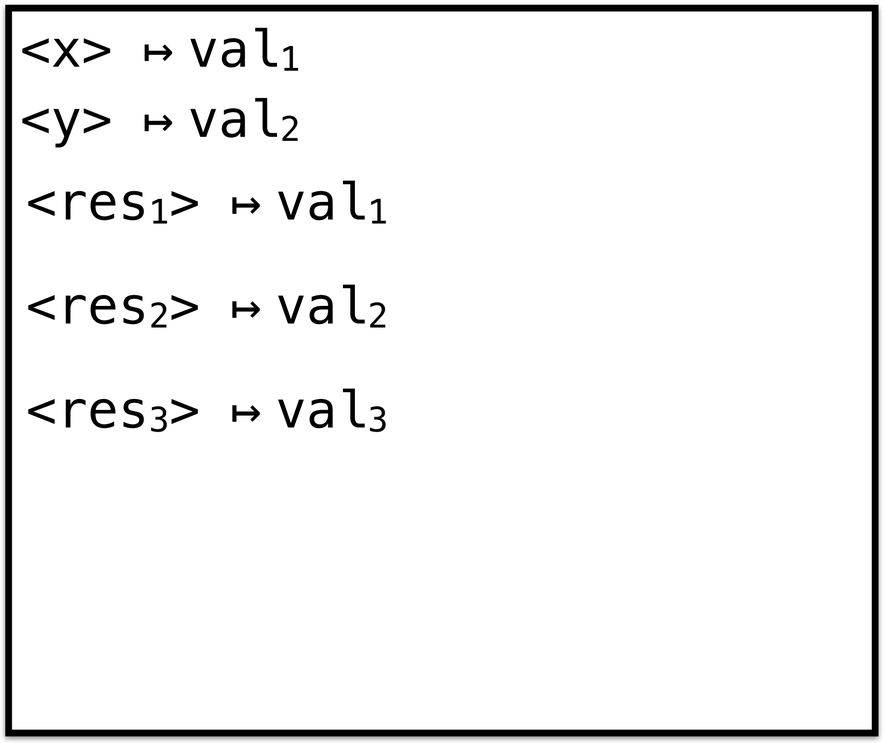
Continuations in function evaluation

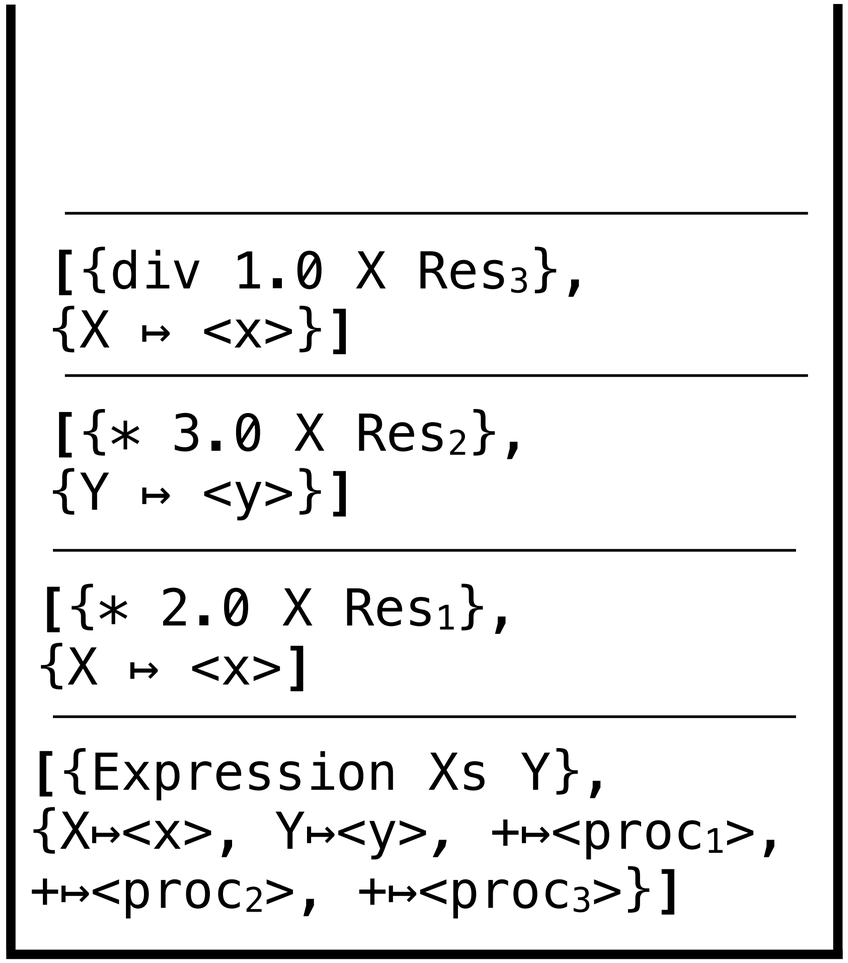
```
fun {Expression X Y}
   2.0*X + 3.0*Y + 1.0 div X + 2.0 div Y
end
```



Continuations in function evaluation

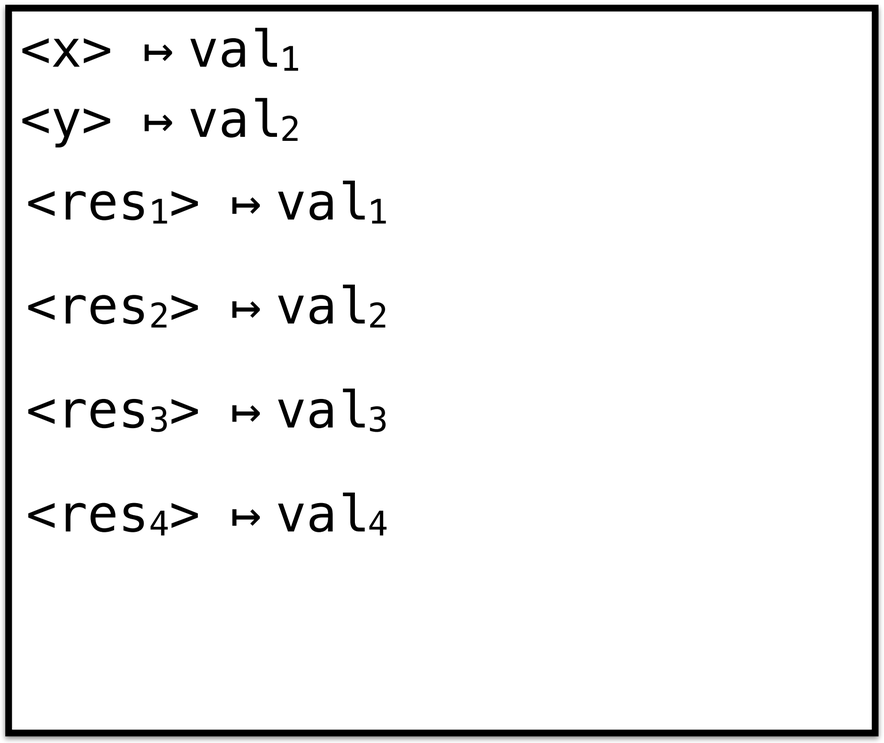
```
fun {Expression X Y}
   2.0*X + 3.0*Y + 1.0 div X + 2.0 div Y
end
```

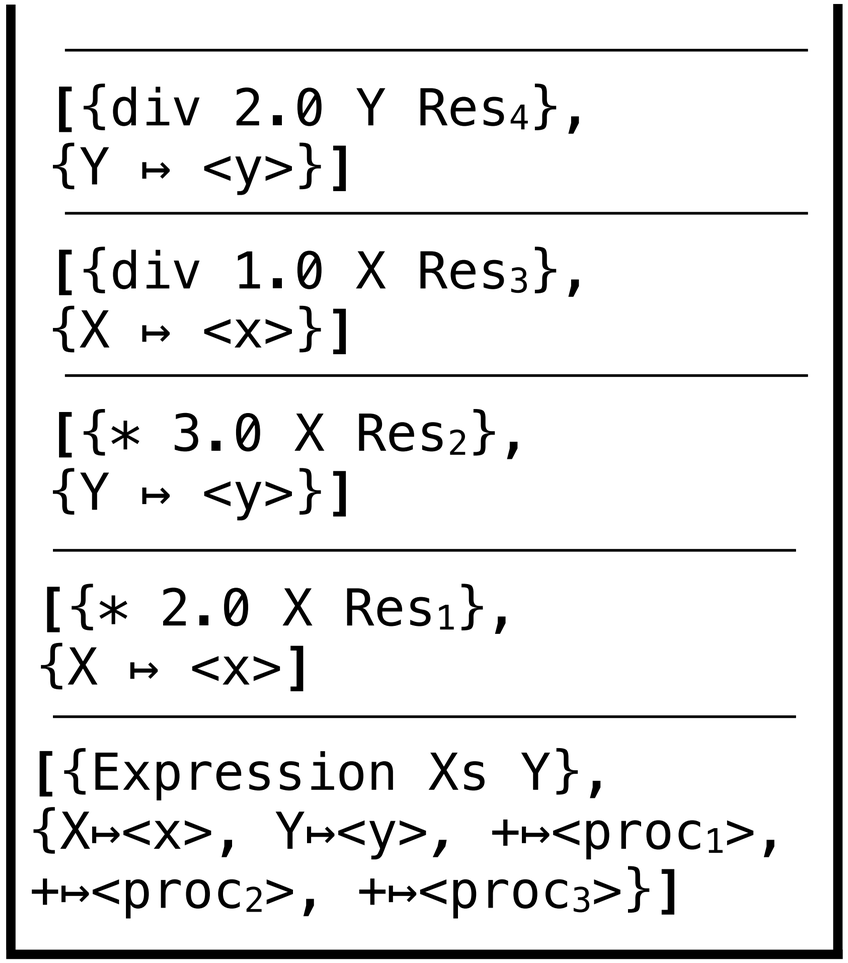




Continuations in function evaluation

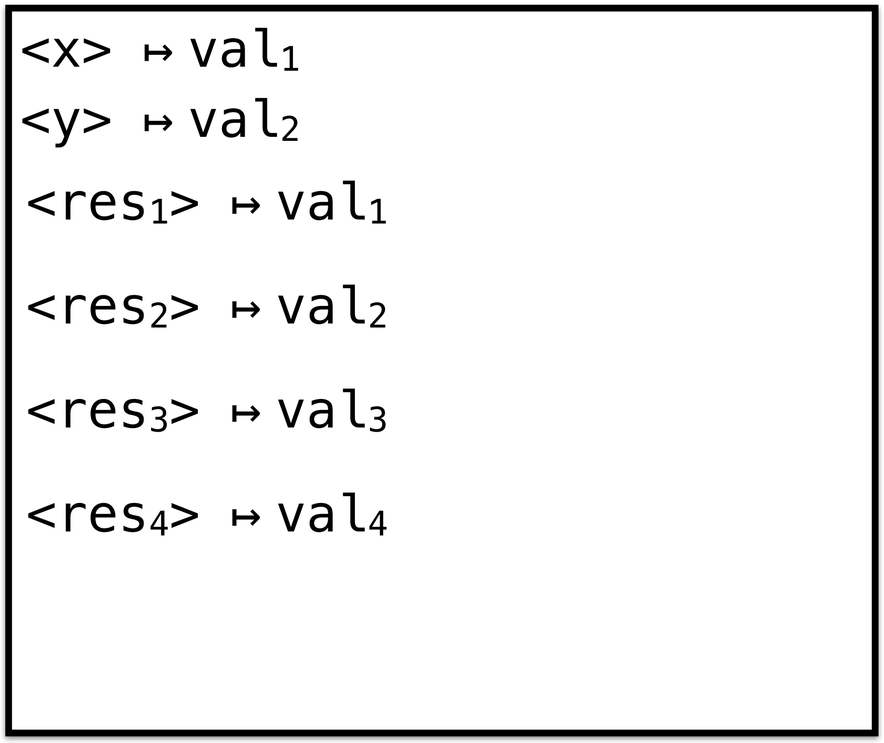
```
fun {Expression X Y}
   2.0*X + 3.0*Y + 1.0 div X + 2.0 div Y
end
```

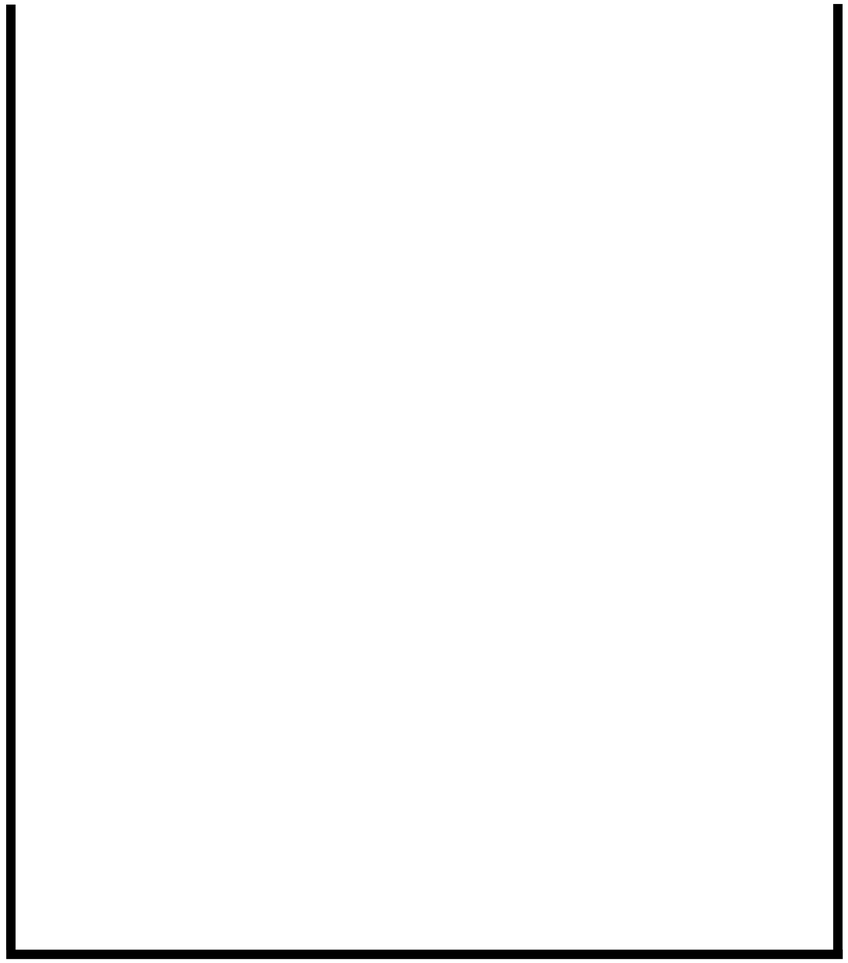




Continuations in function evaluation

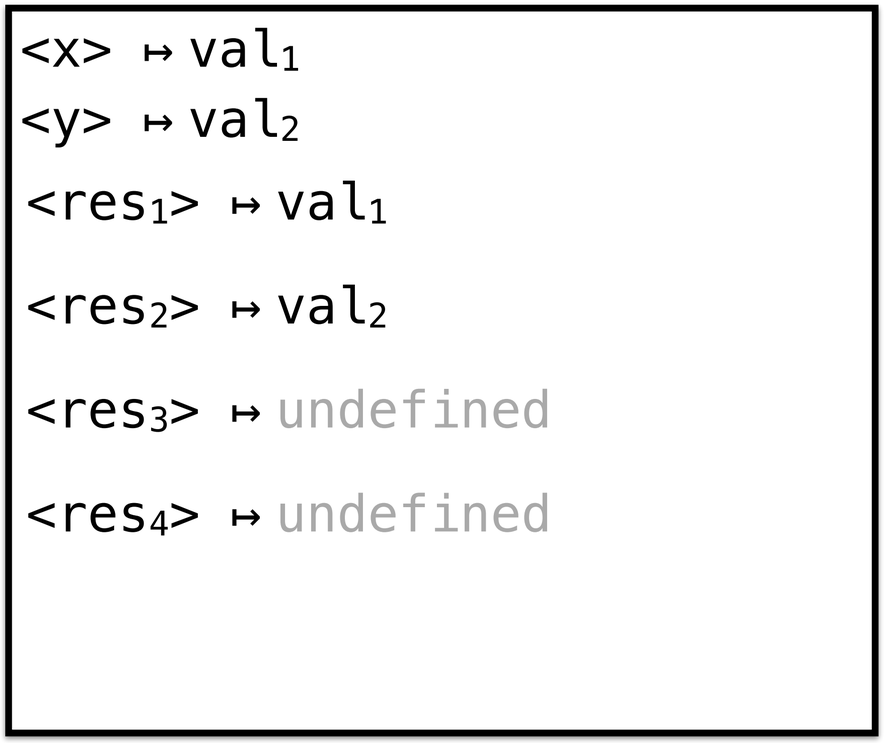
```
fun {Expression X Y}
   2.0*X + 3.0*Y + 1.0 div X + 2.0 div Y
end
```

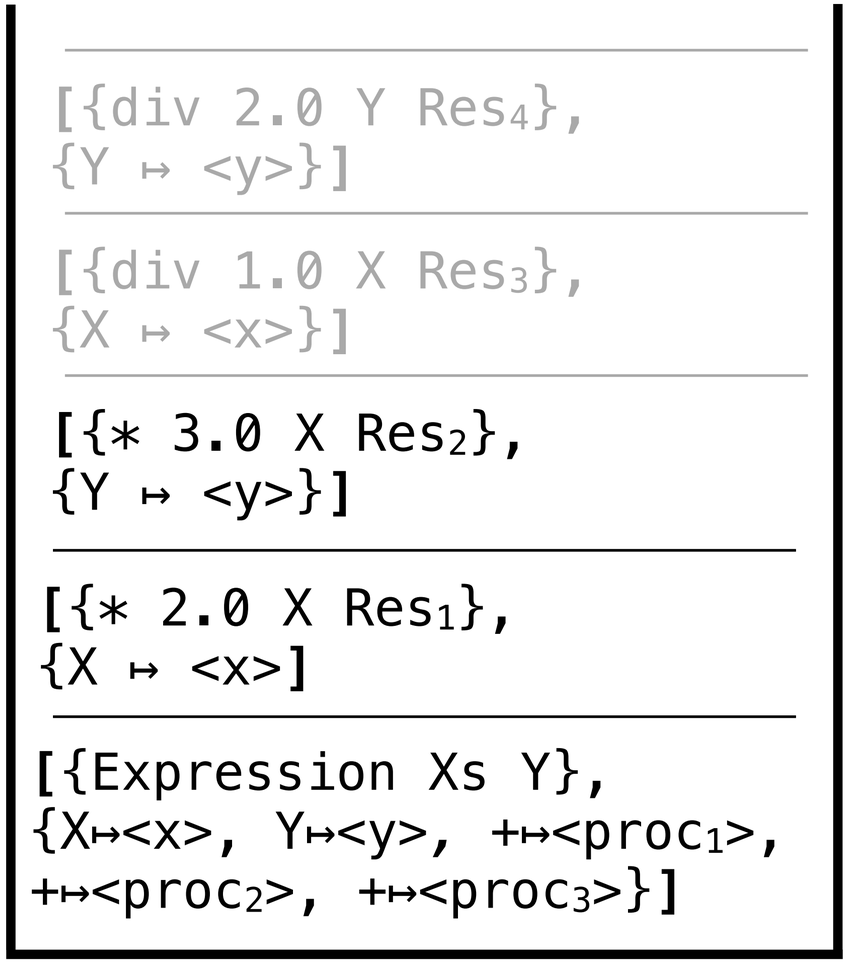




Continuations in function evaluation

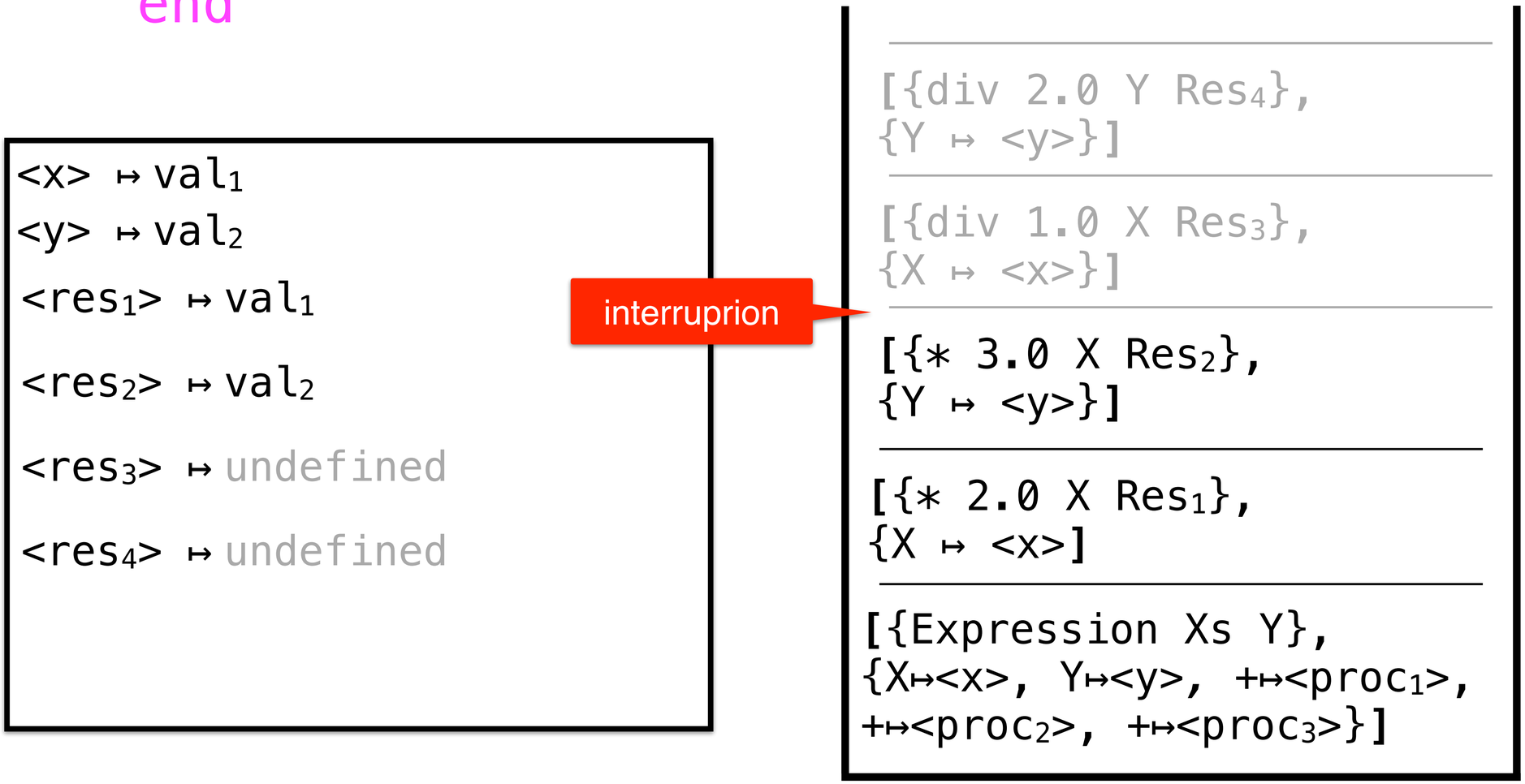
```
fun {Expression X Y}
   2.0*X + 3.0*Y + 1.0 div X + 2.0 div Y
end
```





Continuations in function evaluation

```
fun {Expression X Y}
   2.0*X + 3.0*Y + 1.0 div X + 2.0 div Y
end
```



Continuations

…To continue we need…

1. Remember where we left of

2. Capture the environment of the current execution

3. A pointer to a function to continue the program

Continuations in function evaluation

```
fun {Expression X Y}
   local Before in
    Before = 2.0*X + 3.0*Y
    fun {Continue Q}
     Before + Q + 2.0 div Y
    in Continue{1.0 div X} end
    end
end
```

Continuations in function evaluation

```
fun {Expression X Y}
   local Before in
    Before = 2.0*X + 3.0*Y
    fun {Continue Q}
     Before + Q + 2.0 div Y
    in Continue{1.0 div X} end
    end
end
```

point to continue

Continuations in function evaluation

```
fun {Expression X Y}
   local Before in
    Before = 2.0*X + 3.0*Y
    fun {Continue Q}
     Before + Q + 2.0 div Y
    in Continue{1.0 div X} end
    end
end
```

point to continue

capture the computation environment

Continuations in function evaluation

```
fun {Expression X Y}
   local Before in
      Before = 2.0*X + 3.0*Y
      fun {Continue Q}
         Before + Q + 2.0 div Y
      in Continue{1.0 div X} end
      end
end
```

Continue computation

point to continue

capture the computation environment

Continuations in function evaluation

```
fun {Div Num Den Cont Default}
   if Den == 0 the Default
   else {Cont Num div Den}
end
fun {Expression X Y}
   local Before in
    Before = 2.0*X + 3.0*Y
    fun {Continue Q}
     Before + Q + 2.0 div Y
    in {Div 1.0 X Continue 0} end
    end
end
```

Continuations in function evaluation

```
fun {Div Num Den Cont Default}
   if Den == 0 the Default
   else {Cont Num div Den}
end
fun {Expression X Y}
   local Before in
    Before = 2.0*X + 3.0*Y
    fun {Continue Q}
     Before + Q + 2.0 div Y
    in {Div 1.0 X Continue 0} end
    end
end
```

Error management

Continuations in function evaluation

```
fun {Div Num Den Cont Error}
   if Den == 0 the {Error}
   else {Cont Num div Den}
end
fun {Expression X Y}
   local Before in
      Before = 2.0*X + 3.0*Y
    fun {Continue Q}
      Before + Q + 2.0 div Y
    in {Div 1.0 X Continue Error} end
    end
end
```

Continuations in function evaluation

```
fun {Div Num Den Cont Error}
   if Den == 0 the {Error}
   else {Cont Num div Den}
end
fun {Expression X Y}
   local Before in
      Before = 2.0*X + 3.0*Y
    fun {Continue Q}
      Before + Q + 2.0 div Y
    in {Div 1.0 X Continue Error} end
    end
end
```

Higher-order

Contextual computation with continuations

when the current device user is the owner, only affects the value of that mode.

4.3 Modes of a Procedure

Context-dependent procedures in Flute are expressed in terms of different modes of a modal. For instance, the context-aware calendar application consists of two modes: (i) private agenda mode, and (ii) public agenda mode. The private agenda mode is executed when the owner is using the device, whereas the public agenda mode is executed when another user is using the device. In Flute, we can express such variations of modes as follows.

```
1  ;modal definition
2  (define agenda (modal (current-user)))
3
4  ;shared variables
5  (define date-range          2)
6  (define display-scale       4)
7
8  ;configuration options definition
9  (define config
10   (create-config suspend resume isolated))
11
12 ;mode definition
13 (define show-private-agenda
14   (mode (agenda)
15     (owner? current-user) ;context predicate
16     (config)              ;configuration options
17     (lambda ()
18       (write "private agendas")
19       (set! display-scale 8)
20       (write bg-colour)
21       (scale display-scale)
22       ...
23       (write calendars))))
24
25 ;mode definition
26 (define show-public-agenda
27  (mode (agenda)
28    (not-owner? current-user)
29    (default-config)
30    (lambda ()
31      (write "public agendas")
32      (write bg-colour)
33      ...
34      (write calendars))))
35
36 (agenda)
```

Listing 2. Defining procedure modes

Listing 2 creates a modal `agenda` using the special form `modal` and two modes `show-private-agenda` and `show-public-agenda` using the special form `mode`. As in the case of modal variables, the `current-user` variable in the modal definition, specifies the context source. `date-range` (on Line 5) and `display-scale` (on Line 6) are shared variables that are visible to both modes. The `date-range` value specifies the date range of calendar items to display. The `display-scale` value specifies the scale of the font size of the calendar display. As with modes of a variable, new procedure modes can be dynamically added to a modal on demand. Each mode definition includes a context predicate that must be satisfied throughout the execution of the mode. In the above example, the context predicates for the private and public modes are `(owner? current-user)` and `(not-owner? current-user)`, respectively. To avoid ambiguities, the developer should ensure that context predicates are mutually exclusive.

In addition, a procedure mode definition includes configuration options that specify a strategy for interruption (i.e., `suspend` or `abort`), a strategy for resumption (i.e., `resume` or `restart`) and a strategy for scoping state changes (i.e., `immediate, deferred` or `isolated`). The developer may use the default configuration `default-config` or can define own configuration options using the `create-config` abstraction. The `default-config` specifies the configuration options as `(:p-false suspend :p-true restart :state-changes immediate)`, which implies that when the context predicate is false the execution is suspended, when the context predicate becomes satisfied again the execution is restarted and any state changes are immediately visible. In the above example, the private agenda mode specifies the configuration as `config` that is defined on Line 9 while the public agenda mode uses the default configuration. For conciseness, the above implementation does not include the graphical user interface (GUI) concerns of the calendar application. The screenshot of the context-aware calendar application running in the public mode on an iPad device can be found in the Appendix (Figure 3).

A context predicate specified in each mode plays two roles. First, it is used by the dispatcher to select the applicable mode (dispatching) to execute for the current context of use. Second, it is also used by the runtime to ensure that the mode execution continues to happen in the correct context by continually evaluating the context predicate at every evaluation step of the procedure of body for the mode.

Let us first explain the dispatching process.

4.3.1 Reactive Dispatching of Modes

The execution of modes is initiated by invoking a modal. Invoking a modal requires a dispatching mechanism to select the applicable mode to execute for the current context. For instance, in Listing 2 invoking the agenda modal as `(agenda)`, may execute a private agenda mode procedure or the public agenda mode procedure depending on the current context.

The dispatcher starts by evaluating all context predicates that are associated with the modes that belong to the same modal. The mode whose context predicate evaluates to *true* is scheduled for execution. However, unlike traditional dispatching mechanisms in conventional languages, the dispatching process in Flute does

[bainomugisha et. al. Interruptible context-dependent execution. Onward! 2012]

CONTINUATION PASSING STYLE



Continuation passing

The idea of **continuation passing-style** programming is to allow every function/operation to be **passed a continuation**, and **terminate by calling the continuation**

No need to return to the calling point, just continue

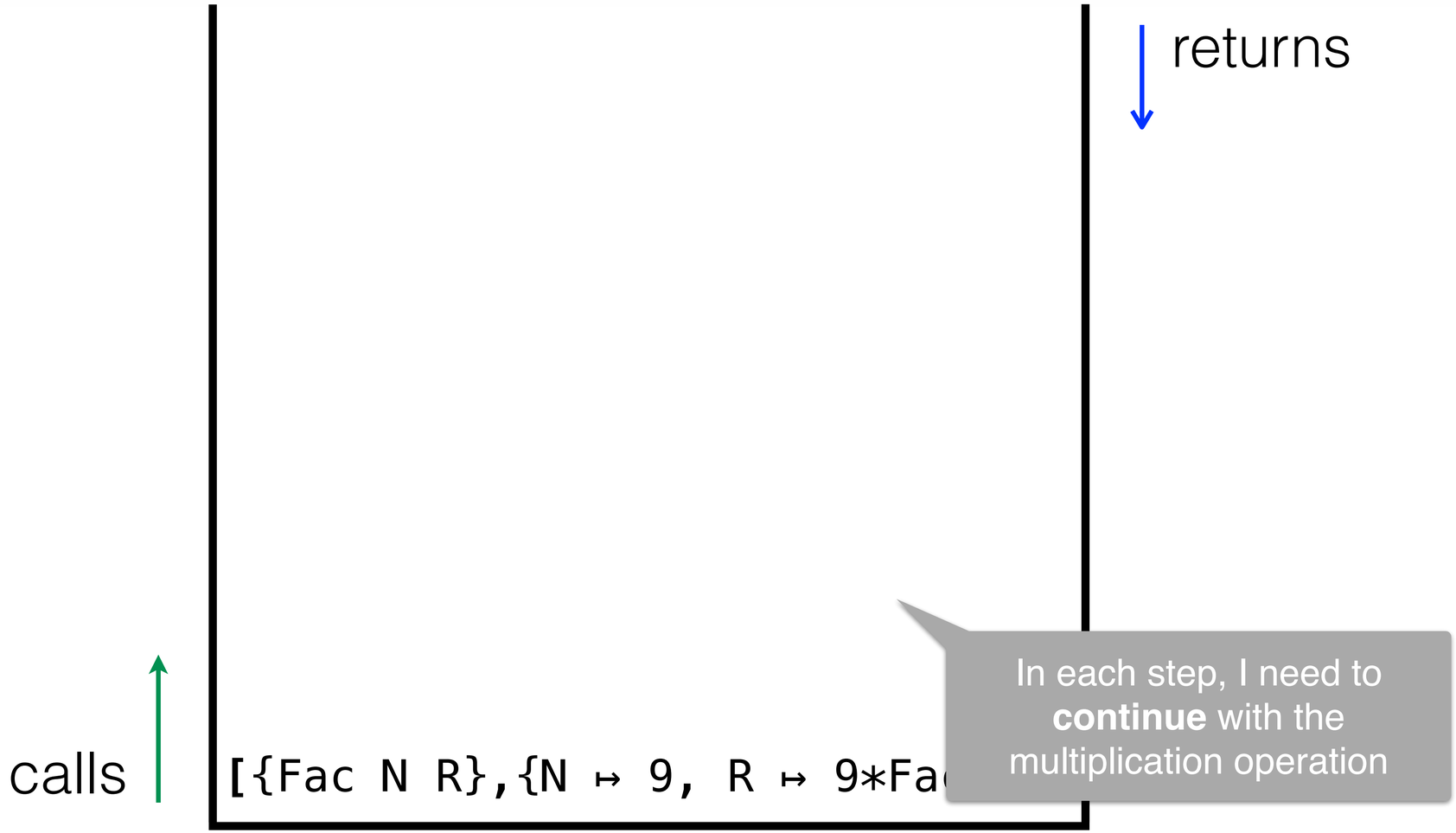
Factorial

Factorial

```
fun {Fact N}
    if N == 1 then 1
    else N * {Fact N-1}
    end
end
```

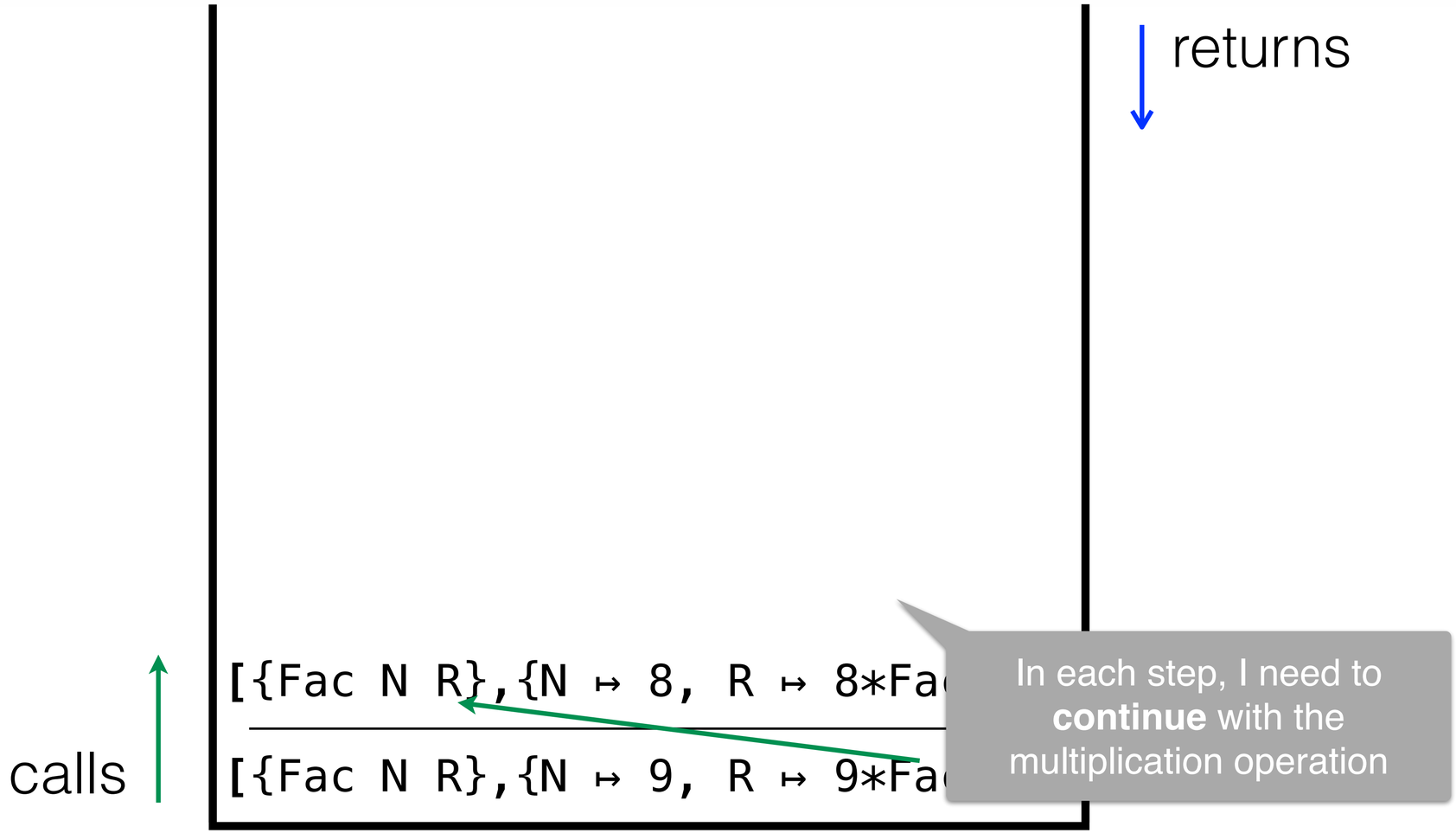
Continuations in Function evaluation

Factorial



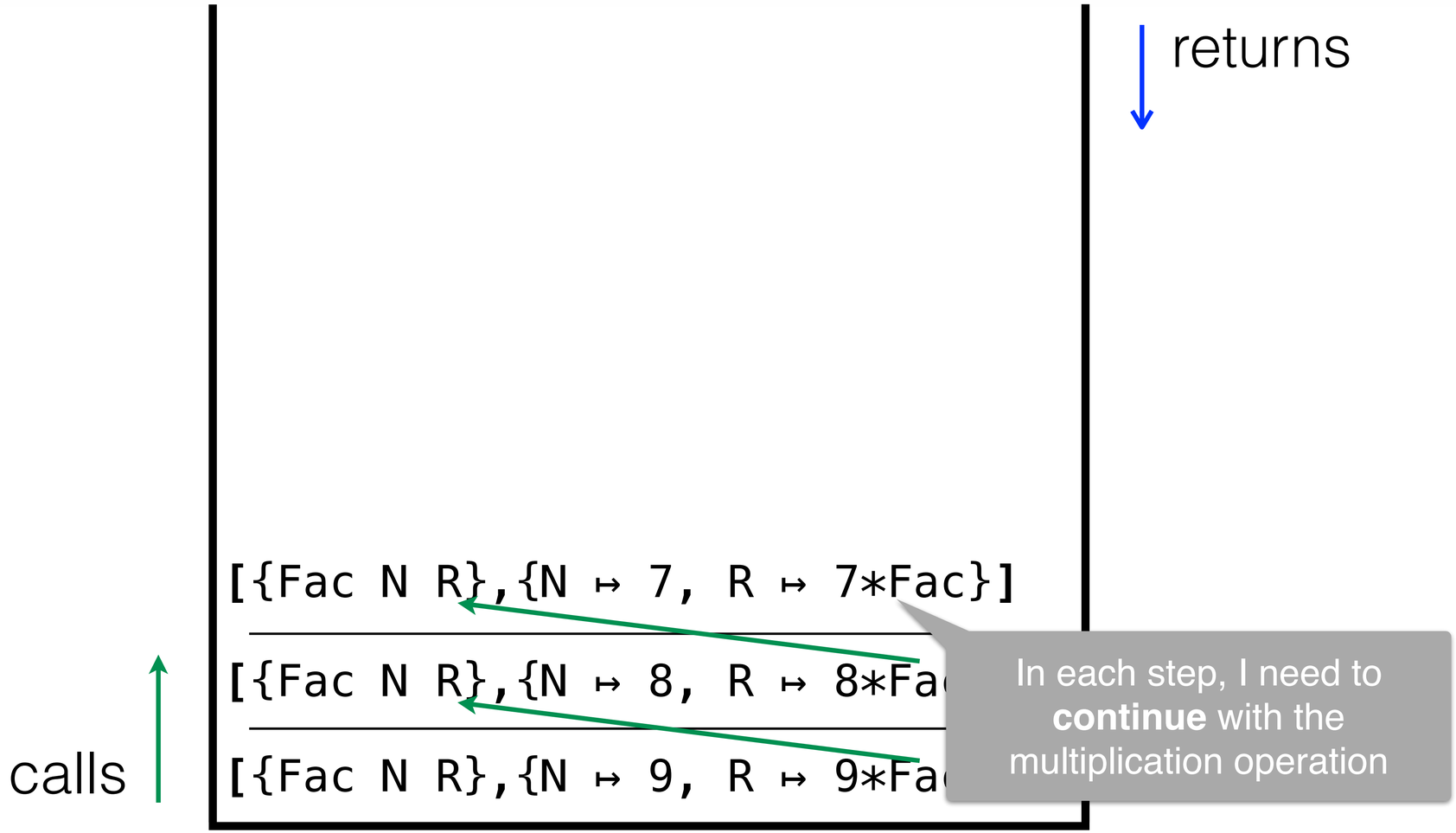
Continuations in Function evaluation

Factorial



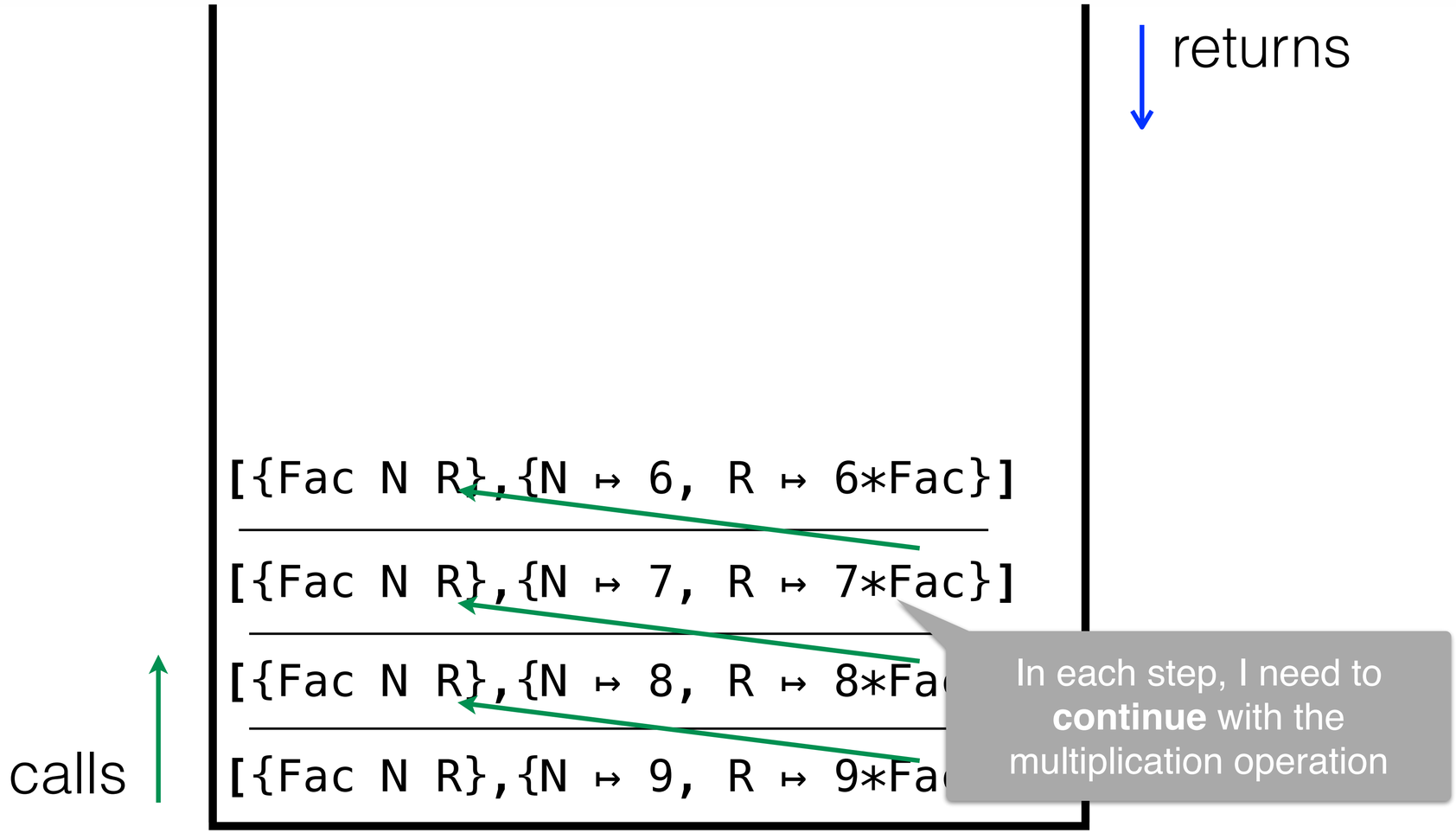
Continuations in Function evaluation

Factorial



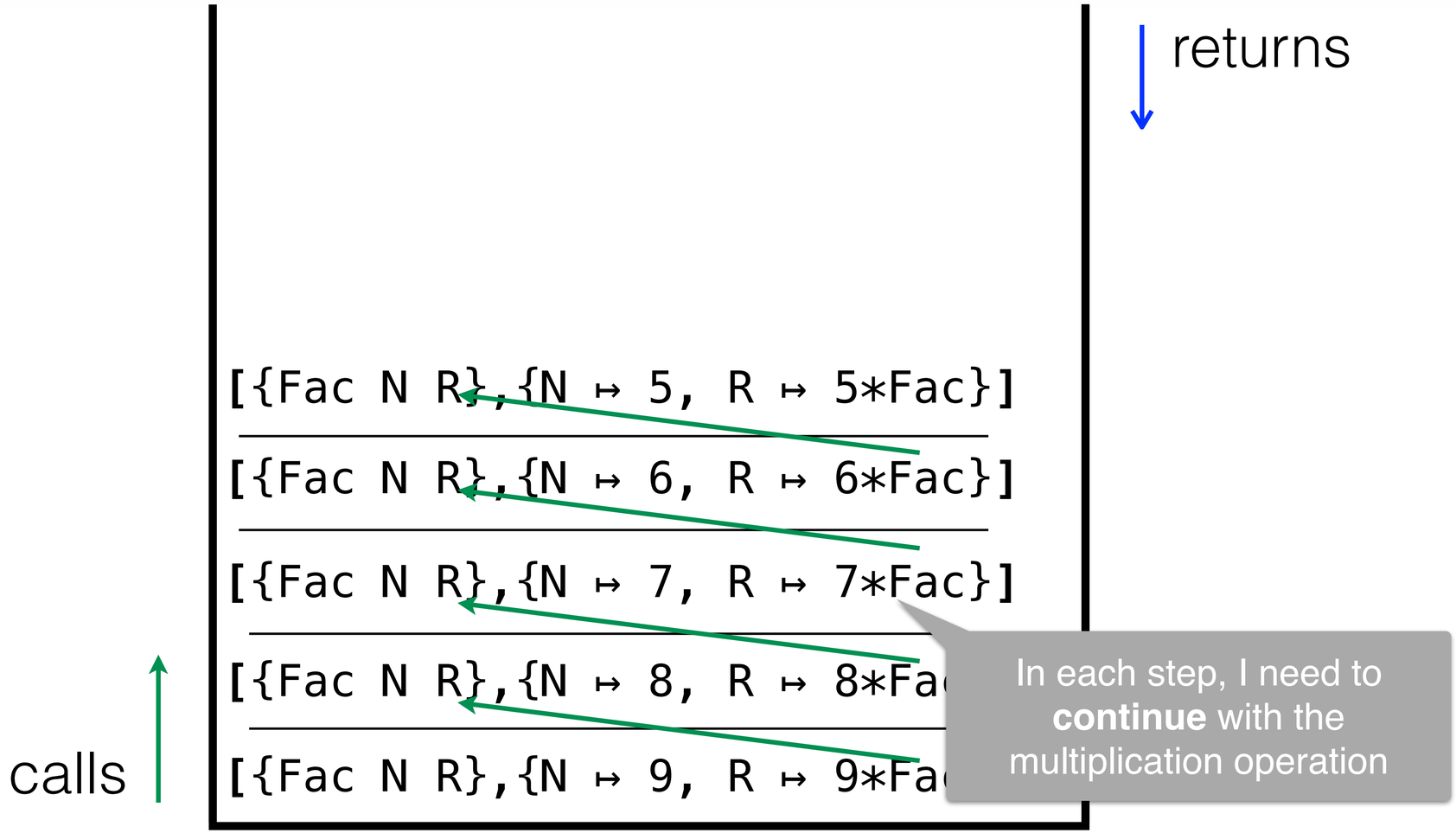
Continuations in Function evaluation

Factorial



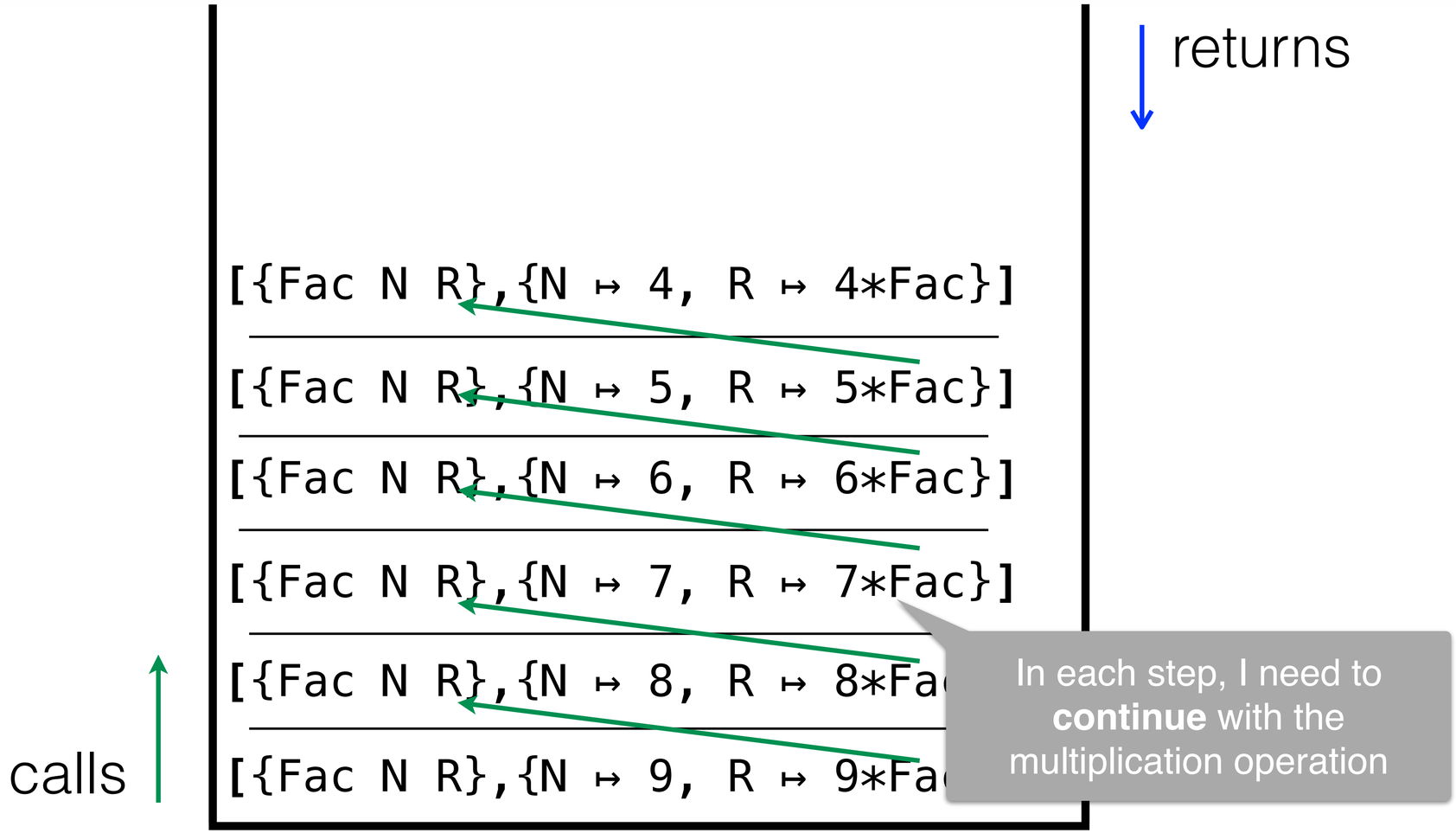
Continuations in Function evaluation

Factorial



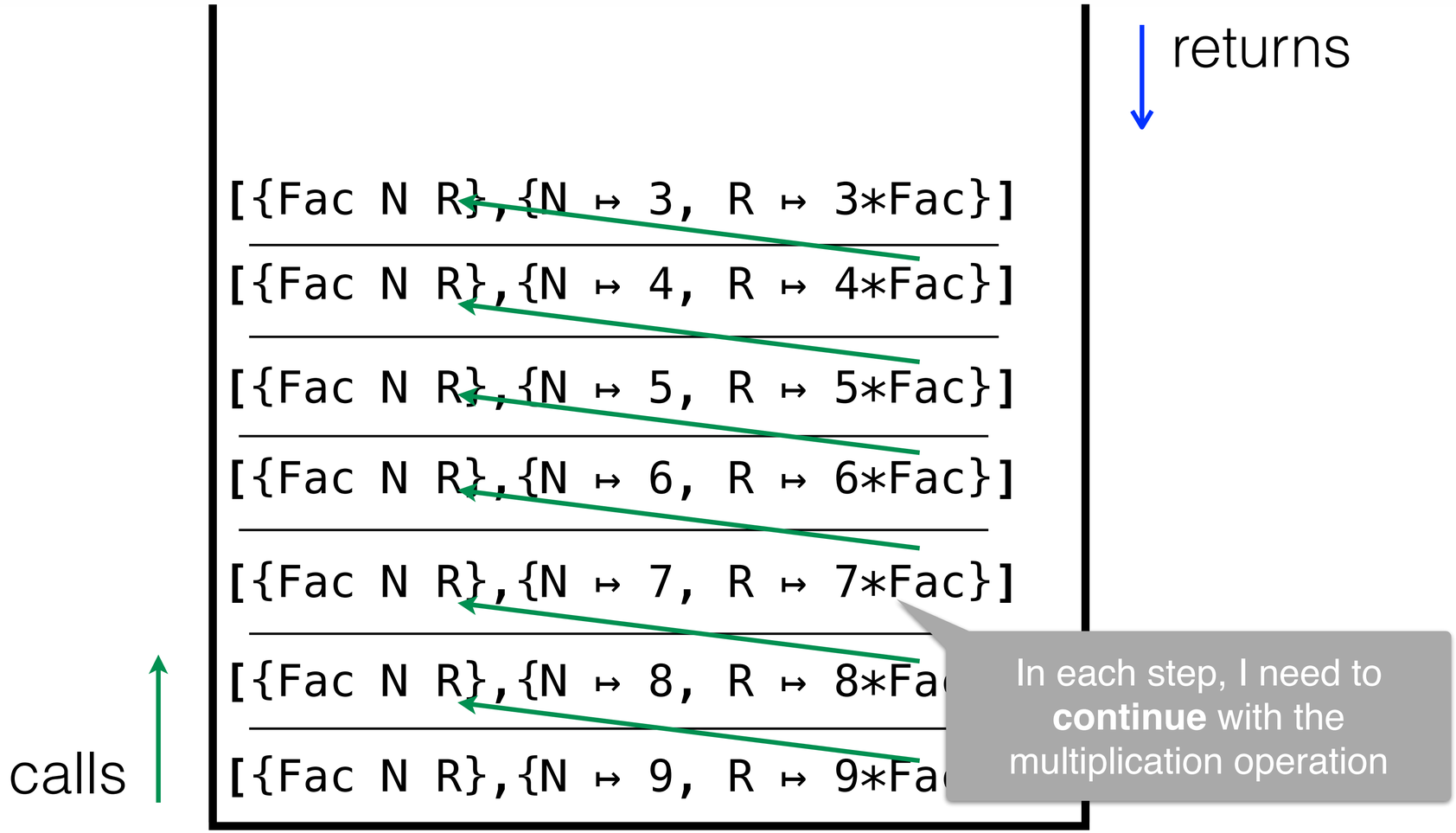
Continuations in Function evaluation

Factorial



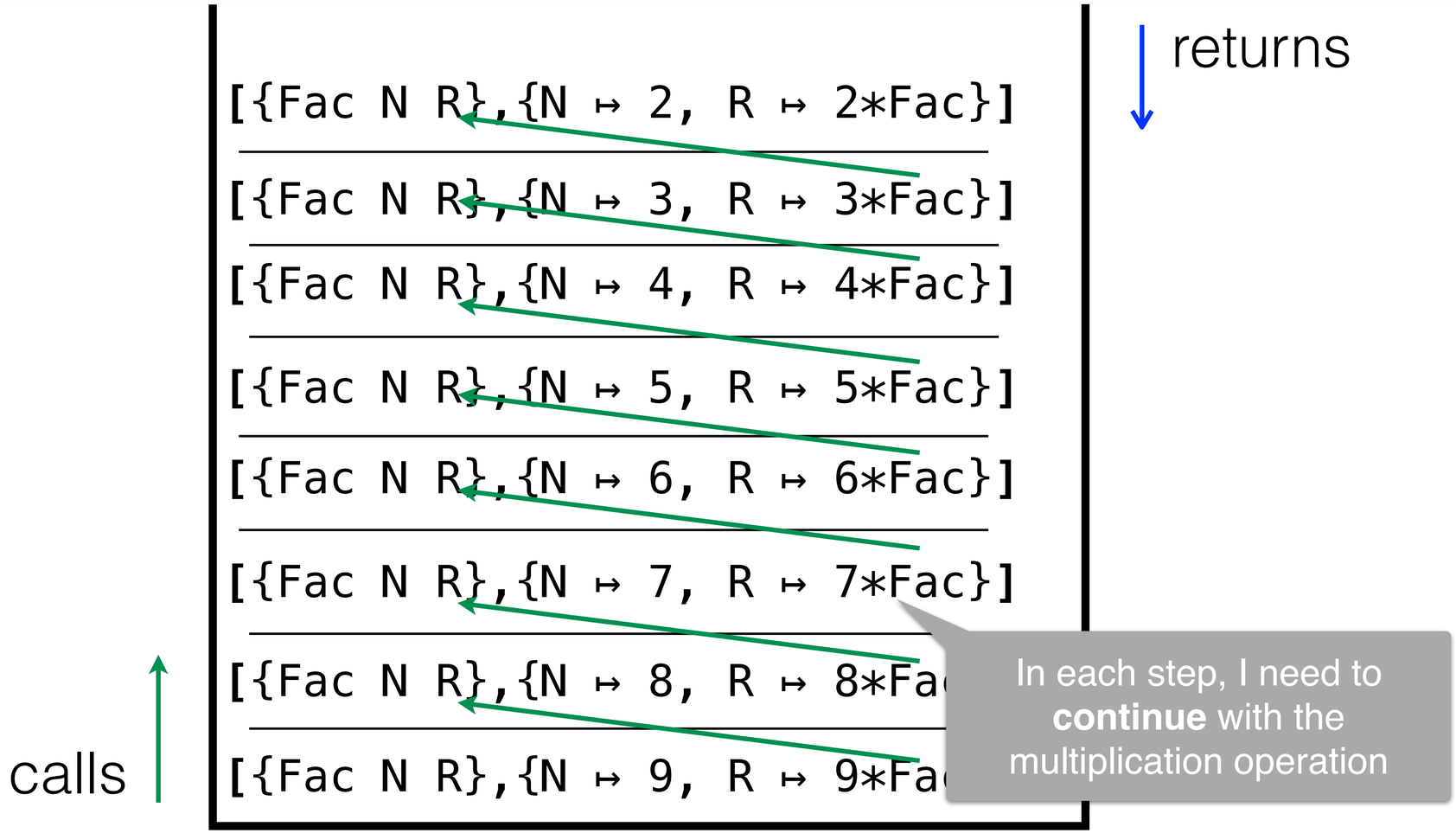
Continuations in Function evaluation

Factorial



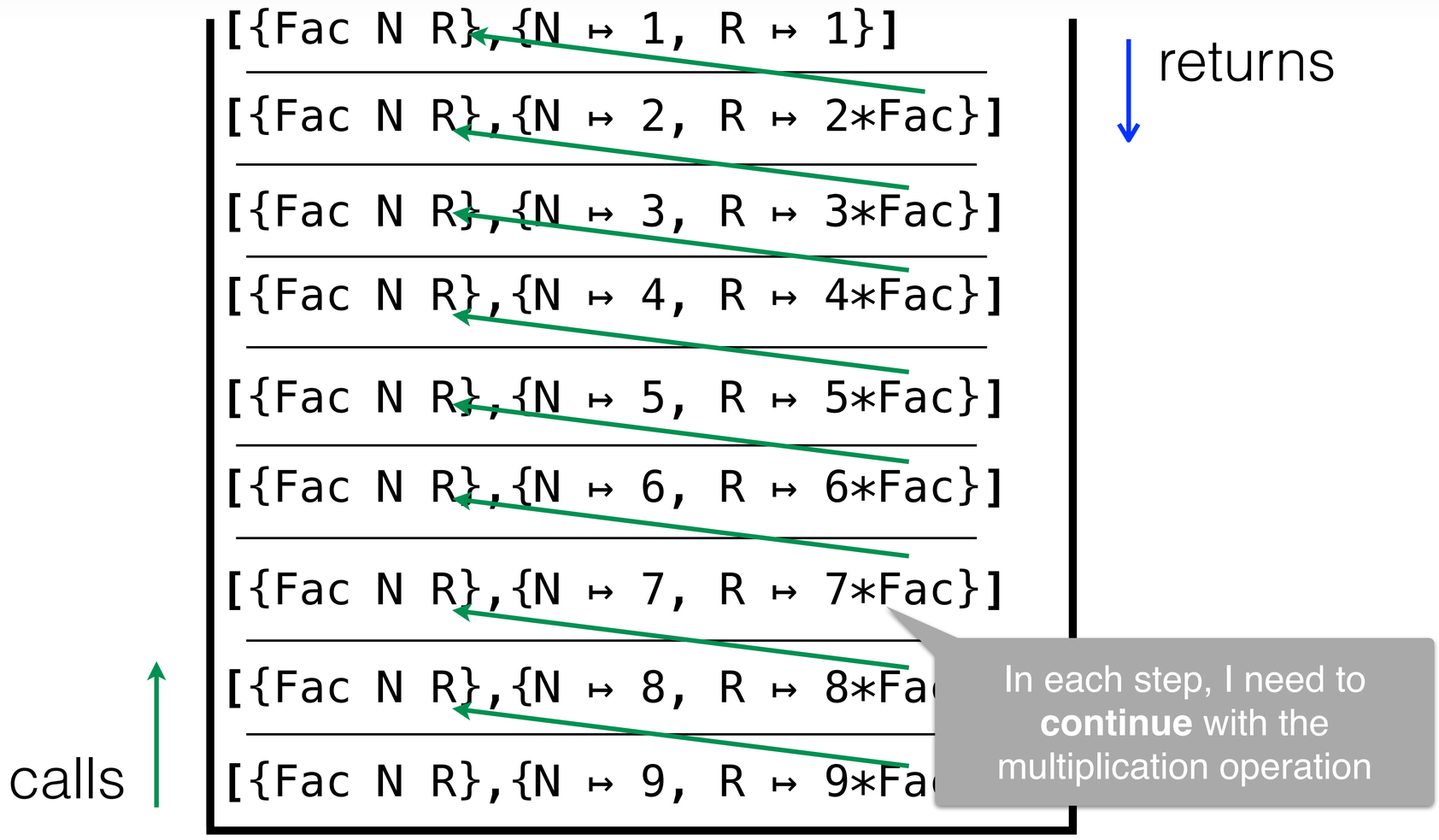
Continuations in Function evaluation

Factorial



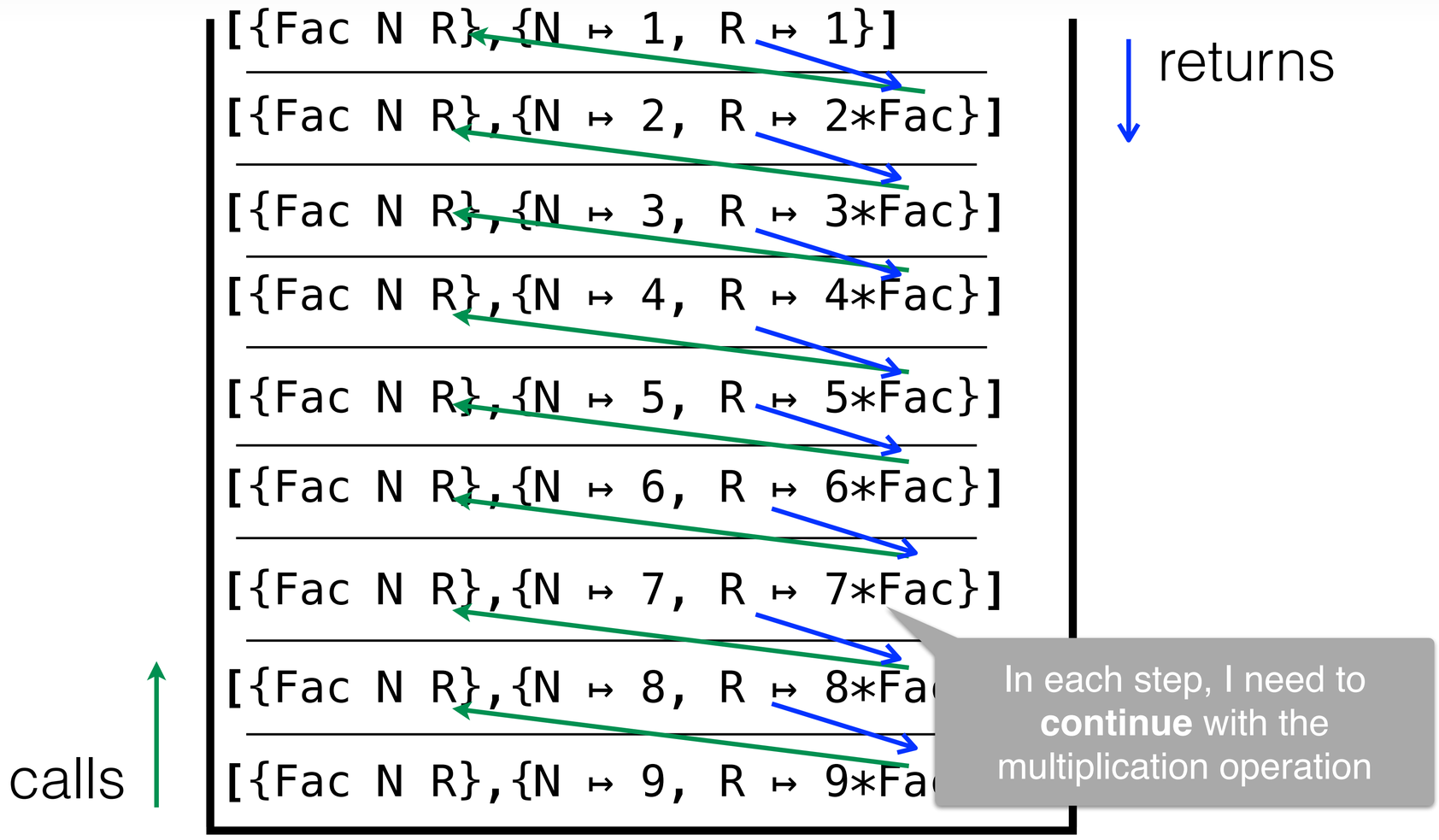
Continuations in Function evaluation

Factorial



Continuations in Function evaluation

Factorial



Continuations in Function evaluation

Factorial

```
fun {ContFact N Cont}
    if N == 1 then {Cont 1}
    else {ContFact N-1 fun {$ X} {Cont N*X} end}
    end
end
```

Continuations in Function evaluation

Factorial

```
fun {ContFact N Cont}
    if N == 1 then {Cont 1}
    else {ContFact N-1 fun {$ X} {Cont N*X} end}
    end
end
```

Initial condition?

Continuations in Function evaluation

Factorial

```
fun {Fact N}
 fun {ContFact N Cont}
    if N == 1 then {Cont 1}
    else {ContFact N-1 fun {$ X} {Cont N*X}
end}
    end
 end
 {ContFact N fun {$ X} X end}
end
```

Continuations in Function evaluation

Factorial

```
  {ContFact 3 fun {$ X} X end}
= {ContFact 2 fun {$ Y} fun {$ X} X end 3*Y end}
= {ContFact 1 fun {$ X} fun {$ Y} 3*Y end 2*X end}
= {fun {$ X} fun {$ Y} 3*Y end 2*X end 1}
= {fun {$ Y} 3*Y end 2}
= 6
```

only moving **forward** in the computation

Continuations & tailrec

Using continuation-passing style, recursive calls become **tail recursive**, however, these are **NOT iterative processes**, as every continuation generates a closure and a new scope for its variables

i.e., space isn't constant!

Continuation passing style

Sum Squares

Do it yourself

Continuation passing style

Sum Squares

Do it yourself

Continuation passing style

Sum Squares

Do it yourself

```
fun {SumSquares Xs}
 fun {ContSums Xs Cont}
    case nil of Xs then {Cont 0}
    [] X|Xr {ContSums Xr fun {$ Y} {Cont X*X+Y} end}
    end
 end
 {ContSums Xs fun {$ X} X+0 end}
end
```

Continuation passing style factorial

Sum Squares

```
  {ContSums [2 3] fun {$ X} X+0 end}
= {ContSums [3] fun {$ Y} fun {$ X} X+0 end 2*2+Y
end}
= {ContSums nil fun {$ X} fun {$ Y} 2*2+Y end 3*3+X
end}
= {fun {$ X} fun {$ Y} 2*2+Y end 3*3+X end 0}
= {fun {$ Y} 2*2+Y end 9}
= 13
```

Tailrec with continuations

```
fun {Grow T}
 case nil of T then nil
 [] leaf(key:K value:V) then
 tree(key:K val:V left:leaf(key:K value:V)
                    right:leaf(key:k value:V))
 [] tree(key:K val:V left:L right:R) then
  tree(key:K val:V left:{Grow L} right:{Grow R})
 end
end
```

Tailrec with continuations

```
fun {Grow T}
 case nil of T then nil
 [] leaf(key:K value:V) then
 tree(key:K val:V left:leaf(key:K value:V)
                    right:leaf(key:k value:V))
 [] tree(key:K val:V left:L right:R) then
  tree(key:K val:V left:{Grow L} right:{Grow R})
 end
end
```

problematic recursion, both calls can’t be tailrec at the same time

Tailrec with continuations

```
fun {GrowCont T Cont}
 case nil of T then nil
 [] leaf(key:K value:V) then
 {Cont K V}
 [] tree(key:K val:V left:L right:R) then
  {GrowCont L fun {$ Tl}
 {GrowCont R fun {$ Tr}
      tree(key:K val:V left:Tl right:Tr) end} end}
 end
end
```

Tailrec with continuations

```
fun {Grow T}
  {GrowCont T fun {$ K V} {MakeLeafTree K V}
 end}
end
```

Tailrec with continuations

Fibonacci

Do it yourself

Tailrec with continuations

Fibonacci

Do it yourself

```
{FibCont 5 fun {$ X} X end}
```

Tailrec with continuations

Fibonacci

Do it yourself

Tailrec with continuations

Fibonacci

Do it yourself

```
proc {FibCont N K}
   if N < 2 then {K 1}
   else
      {FibCont N-1
       fun {FibN-1}
         {FibCont N-2
            fun {FibN-2}
              {K FibN-1 + FibN-2}
            end}
       end}
   end
end
```

Tailrec with continuations

Fibonacci

Do it yourself

```
proc {FibCont N K}
   if N < 2 then {K 1}
   else
      {FibCont N-1
       fun {FibN-1}
         {FibCont N-2
            fun {FibN-2}
              {K FibN-1 + FibN-2}
            end}
       end}
   end
end

{FibCont 5 fun {$ X} X end}
```

Continuation passing style tailrec

Do it yourself

$$\tau(x,y,z) = \begin{cases} \tau(\tau(x-1,y,z),\tau(y-1,z,x),\tau(z-1,x,t)) & \text{if } y < x \\ z & \text{otherwise} \end{cases}$$

Implementing continuations

Continuations from Generalized Stack Inspection *

Greg Pettyjohn
Northeastern University

John Clements
Northeastern University

Joe Marshall
Northeastern University

Shriram Krishnamurthi
Brown University

Matthias Felleisen
Northeastern University

Abstract

Implementing first-class continuations can pose a challenge if the target machine makes no provisions for accessing and re-installing the run-time stack. In this paper, we present a novel translation that overcomes this problem. In the first half of the paper, we introduce a theoretical model that shows how to eliminate the capture and the use of first-class continuations in the presence of a generalized stack inspection mechanism. The second half of the paper explains how to translate this model into practice in two different contexts. First, we reformulate the servlet interaction language in the PLT Web server, which heavily relies on first-class continuations. Using our technique, servlet programs can be run directly under the control of non-cooperative web servers such as Apache. Second, we show how to use our new technique to copy and reconstitute the stack on MSIL.Net using exception handlers. This establishes that Scheme's first-class continuations can exist on non-cooperative virtual machines.

Categories and Subject Descriptors F.3.3 [*Logics and Meanings of Programs*]: Studies of Program Constructs—Control primitives; H.3.4 [*Information Storage and Retrieval*]: Systems and Software—World Wide Web (WWW); I.2.2 [*Artificial Intelligence*]: Automatic Programming—Program transformation; F.3.2 [*Logics and Meanings of Programs*]: Semantics of Programming Languages—Operational semantics

General Terms Languages, Theory

Keywords A-normal form, continuation-passing style, stack inspection, Web programming, continuations, defunctionalization, Scheme

1. Motivation: Continuations, VMs, and the Web

When an interactive Web program issues a query, a non-local transfer of control takes place. It is now the user (possibly an intelligent agent) who is in charge. This user may decide to respond to the query once, twice or many times (or not at all), thus re-launching the rest of the program's computation once, twice, or many times. Many people have therefore concluded that a language with first-class continuations (such as Scheme [19]) is a strong match for implementing this kind of interactive program [11, 18, 26, 33].

Over the past several years, we have explored the continuation model of Web programming in two different directions. Initially, we designed, implemented, and evaluated a Web server that implements the run-time primitives for Web interactions via Scheme's first-class continuations [14]. The investigation validated that this approach is suitable for a variety of situations and even led to a commercial product [20]. Sadly, only programs running on a custom Web server can benefit from this approach.

To address this concern, we also experimented with a variant of the continuation-passing transformation (CPS) for automatically restructuring interactive programs for the Web [13, 21]. In principle, this transformation can be used with a wide range of programming languages and should therefore help Web programmers in many situations. The problem, however, is that the transformation affects the entire program. Since modern Web applications are often written in multiple languages, it is nearly impossible to perform a whole-program transformation of them. Worse, CPS requires tail-call optimization or, in its absence, trampolines, a technique due to Jon L. White [personal communication June 2005], which can be much more expensive. It is therefore economically impossible to use this idea with existing languages and run-time libraries. In short, we are left with the challenge of equipping conventional programming languages with the capabilities of grabbing and reinstalling a continuation even if these languages' run-time organizations do not support such actions.

The very same problem comes up in a different context: the implementation of languages with continuations on virtual machines such as Sun's JVM [34] or Microsoft's CLR [22]. Mirroring traditional programming languages, these machines do not provide instructions for installing and saving the run-time stack. Usually, Scheme-on-VM implementors give up on first-class continuations [1, 2], or they allocate the control stack in the heap of the machine [23]. As Bres, et al. [2, section 2] point out, however, with this second strategy, "it might be expected that . . . JITs are far less efficient on codes that manages their own stack" [sic]. Furthermore, allocating the stack on the heap effectively disguises the stack, hiding it from the rich set of programming tools such as steppers, debuggers, and profilers, as well as contemporary security managers, which expect to find run-time information on the stack.

In this paper, we present a solution to this dilemma, first in the context of a theoretical model and then in the context of two pro-

* This research has been supported by NSF awards to Felleisen and Krishnamurthi as well as a Microsoft donation to Felleisen.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ICFP '05 September 26-28, 2005, Tallinn, Estonia
Copyright © 2005 ACM 1-59593-064-7/05/0009. . . $5.00.

https://www2.ccs.neu.edu/racket/pubs/stackhack4.html

Tailrec when there is no tailrec

Trampolines

Trampolines are just like loops, where **each function** in the loop, *thunk,* **returns the next function to execute**

```
fun {Even N}
   if N == 0 then done(true)
   else {fun {$ } {Odd N-1}} end
  end
end
```

value to “bounce” of the trampoline

“jump” to the next function

http://blog.richdougherty.com/2009/04/tail-calls-tailrec-and-trampolines.html

Tailrec when there is no tailrec

Odd

Do it yourself

Trampolines

Tailrec when there is no tailrec

Odd

Do it yourself

Trampolines

Tailrec when there is no tailrec

Odd

Do it yourself

```
fun {Odd N}
   if N == 0 then done(false)
   else {fun {$ } {Even N-1}} end
  end
end
```

Trampolines

Summary

Accumulators

✓ Accumulators are useful to **turn recursive processes** into **(iterative) tail-recursive**

- If multiple accumulators are used, the **size of parameters** grows quickly and it is too **verbose/hard to manage**

Summary

Continuations

- ✓ Continuations will always allow you to use tail-recursive processes
- ✓ **Continuations are valuable techniques** beyond recursive processes (e.g., super useful for concurrent programming, adaptive systems)

- Generally **not** a **straightforward** conversion
- If not done carefully it can really **hurt performance and readability** of

Summary

Tarampolines

✓ If stack space is scarce, trampolines will save the day; otherwise you'll have to go imperative

- Hard to read and write
- Slower compared to other alternatives